Neural models Lab submission

Input space(X):

(X1,X2)= (0,0),(0,1),(1,0),(1,1)

Output space(Y):

Y=0,1

Examples:

Y(0,0)=0

Y(0,1)=1

Y(1,0)=1

Y(1,1)=0

A single affine model followed by a sigmoid function would introduce non-linearity but it still follows an increasing pattern. The observed output does not uniform increase(or decrease) with the input.

Model specifications:

2 inputs, 1 hidden layer with 2 inputs, 1 output(sigmoid), binary cross entropy loss

Validation:

1. Final loss < 0.05
2. P((0,0))>0.5, P((1,0))<0.5, P((0,1))<0.5, P((1,1))>0.5
3. Output is [0, 1, 1, 0].

Prompt used:

Generate minimal PyTorch code for the following model and dataset: the xor gate with a 2–2–1 network; your chosen hidden activation; a sigmoid binary output (or, preferably in PyTorch, logits with BCEWithLogitsLoss); random weight initialisation; full-batch training for a few thousand lightweight CPU steps;. Do not change the architecture or task. After training, report the final loss, all four probabilities, thresholded labels, and one parameter-gradient tensor. Set a random seed for reproducibility and explain each test in one sentence

Results:

Final loss: 8.336281462106854e-05

Probabilities: [5.931824125582352e-05, 0.9998986721038818, 0.9998774528503418, 5.0744012696668506e-05]

Labels (p>0.5): [0, 1, 1, 0]

Final grad of hidden weight (2x2):

tensor([[2.9268e-06, -3.5339e-06],

[-5.9643e-06, 3.2084e-06]])

Symmetry:

The final probabilities were [0.5, 0.5, 0.5, 0.5] and the labels [0, 0, 0, 0], so only 2 of 4 are correct. The loss sat at ln 2 for all 3000 steps.

- Hidden unit 1 and unit 2 have the same weights, so they compute the same output for every input: $h_1 = h_2$.
- The output layer then sees two identical signals. The gradient reaching each hidden unit is the output error times that unit's outgoing weight, and the two outgoing weights are equal, so both units receive the same gradient.
- The update is W ← W − lr·grad. Two equal rows minus equal updates are still equal, so by induction they remain identical forever. Gradient descent can never break this symmetry, so the two units act as one. In effect the network has a single hidden unit.

Activation:

Hidden activation	**Final loss**	**All 4 correct?**	**Early $\|\|\nabla W_1\|\|$**
sigmoid	0.690306	No	0.001497
tanh	0.002358	Yes	0.054222
ReLU	0.477446	No	0.001611

Reflection questions:

1. Depth vs. nonlinearity. Neither alone is enough. Stacking linear layers adds depth but collapses to one affine map, and a single affine layer plus sigmoid is nonlinear in its outputs but still has a straight decision boundary. XOR needed a hidden layer and a nonlinear activation together. One caveat: I argued the all-linear case analytically and didn't run it. What we did run (zero init) showed the network stuck at 0.5.

2. Evidence of a useful signal. In the successful tanh run (seed 0), the loss fell from 0.7152 to 0.0024, the probabilities moved to about 0.003 / 0.998 / 0.998 / 0.003, and all four labels were correct. A nonzero gradient alone proves none of that. That said, the strongest direct evidence of a *correct* gradient, a finite-difference comparison, is something we never ran. The outcome is good indirect evidence, but not proof.

3. Zero init. With identical weights, both hidden units compute the same output, so they receive the same gradient and get the same update, and the rows stay equal forever. In our run it was worse: all gradients were exactly zero (zero output weights block the hidden gradient, and tanh(0) = 0 blocks the output gradient), so nothing moved. Random init breaks the symmetry.

4. Activation vs. gradient.

- *Engineering observation:* on seed 0 the early first-layer gradient norm was 0.054 for tanh versus about 0.0015 for sigmoid and ReLU, and only tanh converged. But on seed 2, tanh and ReLU had nearly equal early norms (about 0.0048 versus 0.0047) and different outcomes, and tanh failed on seed 1 where sigmoid succeeded.
- *Scientific explanation:* sigmoid's maximum slope is 0.25 and its outputs aren't zero-centred, while tanh has slope 1 at 0 and is zero-centred, so gradients shrink less. ReLU's failures may come from inactive hidden units, which is my hypothesis, not something I checked.

The norm is a loose diagnostic, not an explanation.

5. Output layer and loss. The loss encodes a probabilistic assumption about the target, and the output layer must produce what that loss expects. Binary labels call for one logit with BCEWithLogitsLoss (a Bernoulli likelihood with the sigmoid built in, which is numerically stable). Adding a sigmoid in the model and then using BCEWithLogitsLoss would apply it twice. Multi-class needs logits with CrossEntropyLoss, and regression needs a linear output with MSE. A mismatch gives a wrong objective or poorly behaved gradients.

6. LLM productivity vs. human verification.

- *Productivity:* generating the full experiment scaffolding quickly (training loop, a seed-controlled activation sweep, assertion tests) in one pass.
- *Verification essential:* my first run failed because PyTorch wasn't installed, and I had earlier predicted outcomes (for example, "about 0.5" for the linear model) from reasoning alone. Only real runs confirmed or refuted them, and the one thing I still can't claim is why ReLU got stuck. A person needs to check claims against run output, and to notice that three seeds are anecdotes.

7. Scaling up.

- *Keep:* loss decreasing, finiteness (no NaN/inf), gradient-norm monitoring, held-out accuracy, seed reproducibility, a few-seed sanity sweep, and cheap init checks such as asserting no symmetric duplicate units.
- *Too expensive:* exhaustive finite-difference checks, which need about two forward passes per parameter, and wide seed sweeps or exhaustive input enumeration.

Replace them with finite-difference checks on a tiny copy of the model or a random subset of parameters, and with directional-derivative checks.